Cross-Domain Fuzzing Through Structure-Aware Mutation and Bandit-Guided Adaptation

DAKSHIN, NEEL, and SAMYAK, IIT Guwahati, India

Software systems spanning file parsers, multimedia engines, and asynchronous network servers present diverse input domain barriers for coverage-guided fuzzers. In the drogon web framework, an uncaught filesystem exception inside a method marked `noexcept` triggered process aborts on oversized upload filenames. We present QTFuzz (Quick Transfer Fuzzer), an adaptive fuzzing architecture combining structure-aware mutation, an independent multi-armed bandit scheduler (Discounted-UCB), and headless in-memory execution shims. We designed a base structural mutator for fastgltf, then extended the architecture to multimedia rendering in raylib via software rasterizer shims, and to asynchronous network protocols in drogon via in-memory socket mocks. Across 59 target binaries, QTFuzz executed 807,520,128 inputs, reaching up to 53,732 executions per second on network parsers without operating system socket bottlenecks. QTFuzz uncovered 16 security defects, with six upstream-confirmed vulnerabilities resulting in two merged pull requests and four maintainer-committed patches. Our results show that domain-specialized in-memory shims and non-stationary bandit adaptation enable effective fuzzing across heterogeneous software domains.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Security and privacy** → *Software security engineering*.

Additional Key Words and Phrases: Coverage-guided fuzzing, Multi-armed bandits, Headless shims, Memory safety, glTF, Multimedia, Asynchronous web frameworks

1 Introduction

Suppose you need to test a C++ software stack that accepts inputs from multiple channels. One component parses 3D binary assets, a second renders graphics and decodes audio, and a third accepts HTTP requests and file uploads over the network. Developers commonly run coverage-guided fuzzers such as libFuzzer [Serebryany 2016] or AFL++ [Fioraldi et al. 2020] on individual functions.

These tools work well on standalone byte parsers. They mutate raw input buffers and track branch coverage using compiler instrumentation. However, these tools stop when software interacts with stateful runtime environments, external graphics displays, or asynchronous network event loops. Traditional fuzzers either require writing bespoke mock harnesses for every component or suffer from low execution throughput when executing full integration builds.

We focus on the problem of building a unified adaptive fuzzing architecture that scales across diverse software layers. Starting from a pure structured file format parser, our goal is to extend the same core mutator and scheduler to multimedia runtimes and asynchronous web frameworks without losing execution speed or bug-finding capability.

Consider the defect in Figure 1, discovered by our fuzzer in the drogon C++ web framework. The method `HttpFileImpl::save` was declared `noexcept`. When an application saves an uploaded file using a client-controlled filename exceeding the filesystem limit (255 bytes on Linux), the call to `std::filesystem::exists` threw a `std::filesystem_error`. Because the enclosing method was marked `noexcept`, the C++ runtime intercepted the uncaught exception and immediately invoked `std::terminate()`. A single malformed upload killed the entire multi-threaded web server process. Across the three evaluated codebases, we uncovered 16 security and reliability defects spanning memory corruption, buffer overflows, and state desynchronization.

```
int HttpFileImpl::save(const std::string &p) const noexcept {
    if (!std::filesystem::exists(p)) // Throws on len > 255
        std::filesystem::create_directories(p);
} // Escaping exception in noexcept invokes std::terminate()
```

Fig. 1. Motivating defect in drogon (**D1**, simplified). Calling throwing `std::filesystem::exists` inside a `noexcept` method causes `std::terminate()` to abort the process when an upload filename exceeds 255 bytes.

Prior work on structure-aware fuzzing handles complex file grammars through custom mutators [Aschermann et al. 2019]. However, existing grammar fuzzers do not provide headless shims for interactive game engines or asynchronous network protocols. Conversely, network fuzzers such as AFLNet [Pham et al. 2020] send packets over real operating system sockets, which restricts throughput to hundreds of executions per second.

We present QTFuzz, an adaptive fuzzing architecture designed to test heterogeneous C and C++ software stacks. QTFuzz operates across four core components: (i) an adaptive mutation scheduler governed by a Discounted-UCB multi-armed bandit algorithm (Section 4.2); (ii) structured mutation extended from binary assets to multimedia rendering through headless shims (Section 4.3); (iii) in-memory TCP socket mocks for asynchronous network servers (Section 4.3); and (iv) independent coverage accounting via GNU linker symbol wrapping (Section 4.4).

We evaluated QTFuzz across three software projects: the fastgltf 3D parser [Spnda 2024], the raylib game engine [Santamaria 2024], and the drogon asynchronous web framework [Tao 2024]. Across 59 target binaries, the tool executed 807,520,128 campaign iterations. QTFuzz discovered 16 security vulnerabilities and defects, resulting in 6 upstream-confirmed defects (two merged pull requests and four maintainer commits) alongside 10 unfiled medium- and high-severity findings. Our results show that modular headless shims allow an in-memory fuzzer to reach up to 53,732 executions per second on network parsers while discovering real-world vulnerabilities.

Contributions.

- Cross-domain fuzzer adaptation across file formats, multimedia runtimes, and network services (Section 3).
- The QTFuzz architecture combining structure-aware mutation, Discounted-UCB scheduling, and linker wrapping (Section 4).
- Domain-specific headless shims for software rasterizers, audio streams, and asynchronous event loops (Section 4).
- Evaluation across three open-source systems over 807M executions documenting 16 defects (Section 5), with open artifacts (Section 8).

2 Background

Coverage-Guided Fuzzing. Coverage-guided fuzzing evaluates software by feeding mutated inputs into a target function while monitoring program execution. Compilers such as Clang instrument basic blocks with feedback counters. The fuzzing engine, such as libFuzzer [Serebryany 2016], retains inputs that trigger new execution paths or edge transitions. AddressSanitizer (ASan) [Serebryany et al. 2012] and UndefinedBehaviorSanitizer (UBSan) detect memory safety violations, out-of-bounds array reads, and integer overflows at the point of failure.

Multi-Armed Bandit Scheduling. An adaptive mutator must decide which operator to apply to a given input. This decision maps directly to the *multi-armed bandit problem* [Auer

et al. 2002]. Given K mutation operators (arms), the scheduler balances exploitation (allocating pulls to operators that yielded new edges) with exploration (trying less frequent operators). In coverage fuzzing, rewards decay over time as the input corpus saturates, creating a non-stationary reward distribution. We adopt Discounted-UCB [Garivier and Moulines 2011], which discounts past observations with parameter $\gamma \in (0, 1)$. For arm i at time step t, the discounted pull count is $n_i(\gamma) = \sum_{s=1}^{t} \gamma^{t-s} \mathbb{I}\{A_s = i\}$. The discounted mean reward is $\bar{r}_i(\gamma) = \frac{1}{n_i(\gamma)} \sum_{s=1}^{t} \gamma^{t-s} R_s \mathbb{I}\{A_s = i\}$, where $R_s \in [0, 1]$ indicates whether step s uncovered a new branch edge. The arm score is:

$$score_i = \bar{r}_i(\gamma) + 2C \cdot \sqrt{\frac{\ln N(\gamma)}{n_i(\gamma)}} \tag{1}$$

where $N(\gamma) = \sum_{j=1}^{K} n_j(\gamma)$ and $C > 0$ governs exploration.

Structured 3D Asset Formats. The glTF 2.0 standard [The Khronos Group 2021] stores 3D scenes as structured JSON metadata paired with binary buffers. The binary GLB container consists of a 12-byte header followed by chunk entries. Parsers such as fastgltf decode JSON elements (nodes, meshes, accessors) and map byte offsets into binary vertex buffers. Fuzzing such formats requires maintaining structural coherence; purely random byte flips fail at early syntax verification stages.

Multimedia Engines. Game engines such as raylib integrate graphics, audio streams, and event serialization in C. Testing these components requires simulating display contexts and audio buffers in memory without physical hardware.

Asynchronous Web Architectures. Modern C++ web servers such as drogon handle high-concurrency HTTP traffic through asynchronous I/O. The networking core uses thread-bound event loops that manage socket buffers and callback chains. Testing these parsers requires driving event loops synchronously in memory to avoid operating system socket latency.

3 Problem Statement

A complex software system consists of multiple subsystems that receive inputs across distinct domains: structured file formats ($\mathcal{D}_{\text{file}}$), multimedia memory streams ($\mathcal{D}_{\text{media}}$), and asynchronous network packets ($\mathcal{D}_{\text{net}}$).

Each domain presents a distinct impedance mismatch for coverage-guided fuzzers:

- **Syntactic Barriers ($\mathcal{D}_{\text{file}}$):** Formats such as glTF reject unstructured inputs during header validation. Mutators must produce structurally valid inputs to reach deep decoding logic.
- **Environmental Dependencies ($\mathcal{D}_{\text{media}}$):** Graphics and audio libraries require display windows, framebuffers, and audio devices. Standard entry points stall on display servers and audio drivers.
- **Asynchronous Statefulness ($\mathcal{D}_{\text{net}}$):** Network frameworks process streams via non-blocking sockets and callbacks. Real network interfaces incur TCP handshake overhead and socket exhaustion.

Formulation. Let $\mathcal{M} = \{m_1, m_2, \ldots, m_K\}$ represent a shared library of mutation operators. For a target harness t, the execution of an input generates a trace covering a set of branch edges $E(t) \subseteq \mathcal{E}$. Given a wall-clock testing budget B, the goal of the adaptive

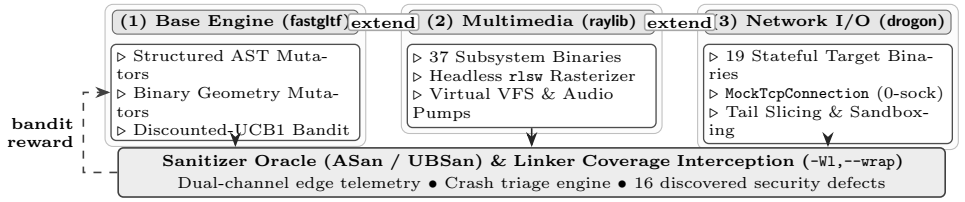


Fig. 2. Overview of QTFuzz. The architecture combines a shared mutation engine and Discounted-UCB bandit scheduler with domain-specific headless shims for 3D assets (①), multimedia rendering (②), and asynchronous network services (③). Coverage rewards update the bandit.

selection policy $\pi : \mathcal{H} \to \mathcal{M}$ is to maximize discovered edge coverage:

$$\max_{\pi} |E(t)| \quad \text{subject to} \quad \text{Time}(t) \leq B \tag{2}$$

where $\mathcal{H}$ denotes the execution and reward history.

Why not write independent custom fuzzers for each domain? Building independent fuzzers for each domain duplicates mutation logic and prevents code reuse. A mutation operator that uncovers integer boundaries in 3D vertex buffers is equally effective at finding Content-Length parsing flaws in HTTP request headers. QTFuzz provides a shared operator framework across domains, while instantiating bandit schedulers per target to dynamically learn domain-specific operator preferences.

Why not test network targets through real operating system sockets? Network fuzzers that communicate over loopback TCP sockets incur operating system scheduling, packet serialization, and kernel socket buffer allocations. When a fuzzer establishes fresh TCP connections per test input, socket allocation and `TIME_WAIT` exhaustion cap throughput to approximately 240 executions per second per core. Even with persistent keep-alive connections, loopback throughput peaks below 1,900 executions per second due to kernel context switching. In contrast, driving parser callbacks directly in memory via `MockTcpConnection` reaches up to 53,732 executions per second on the same parsing logic.

4 Approach

Figure 2 presents the architecture of QTFuzz. The core concept is a shared mutator and bandit scheduling framework paired with domain-specialized headless in-memory shims. We do not claim standard compiler instrumentation, libFuzzer's in-process loop, or AddressSanitizer checks as new. Our design contribution lies in the cross-domain fuzzer adaptation, the non-stationary bandit scheduler, and the zero-socket in-memory network mocking layer.

4.1 Shared Mutation Engine and Operators

QTFuzz provides a unified library of mutators grouped into four operational arms:

- **Byte Mutation (`bin.libfuzzer`):** Applies bit flips, byte insertions, and chunk deletions.
- **Integer Boundary Injection (`bin.intedge`):** Overwrites 2-, 4-, and 8-byte integers with boundary values $(0, \pm 1, 0\text{x}7\text{F}, 0\text{xFFFF}, 2^{31} - 1, \texttt{SIZE_MAX})$.
- **Block Duplication (`bin.blockdup`):** Duplicates repeated structures (glTF nodes, image scanlines, HTTP headers) to test buffer limits.
- **Length and Offset Fixup (`bin.fixup`):** Repairs container sizes (RIFF, GLB, Content-Length) with 50% probability to exercise inner parsers.

4.2 Discounted-UCB Bandit Scheduling

To steer mutation dynamically, QTFuzz implements Discounted-UCB (Equation 1) with discount factor $\gamma = 0.95$ and exploration parameter $C = 0.5$. Because code coverage saturates over time, new edge discoveries decay, rendering stationary bandit assumptions invalid. The discount factor gives higher weight to recent discoveries, preventing early-stage reward plateaus from permanently starving exploratory arms.

Attribution Mechanism. In libFuzzer's in-process loop, the custom mutator hook generates an input x_t using selected arm a_t. The harness immediately executes the target function with x_t. Before the subsequent mutator call, the coverage interception layer checks whether any previously unseen basic block was tripped during the execution of x_t. If at least one new edge was discovered, the fuzzer registers a positive binary reward $R_t = 1$; otherwise $R_t = 0$. The discounted reward $\bar{r}_{a_t}(\gamma)$ and pull count $n_{a_t}(\gamma)$ are updated before selecting arm a_{t+1}.

4.3 Domain-Specialized Headless Shims

To test stateful libraries without physical displays or kernel sockets, QTFuzz provides domain-specific in-memory shims. Developing these shims required moderate manual engineering effort (4,350 lines of C/C++): while the shims must be written once per framework, they connect to the common mutation and scheduling core.

Multimedia Virtual Machines (raylib). Testing interactive graphics through a single entry point causes mutators to stall on display setup. We decomposed raylib into 37 focused targets using two virtual machine designs:

- **Image VM (`fuzz_image_ops`):** Executes 52 image operations (resampling, cropping, convolution kernels) via a deserializing reader (`fz_rd`).
- **Render VM (`fuzz_render_vm`):** Drives 48 drawing and state operations directly against the software rasterizer (`rlsw`) over memory framebuffers, testing matrix stacks and geometry without display servers.

For audio, harnesses pump the mixer callback synchronously on the fuzzer thread, eliminating race conditions.

In-Memory Network Mocks (drogon). Web framework fuzzing is traditionally throttled by TCP socket allocation. We decoupled drogon's parser logic from the operating system via three shims:

- `MockTcpConnection`**:** Implements `trantor::TcpConnection` in memory, pushing buffers directly into HTTP callbacks without socket system calls.
- **Tail Slicer:** Uses trailing 4 bytes of each input as a chunk delivery schedule, testing packet fragmentation without corrupting seed payloads.
- **RAII Sandboxing:** Executes multipart upload targets in temporary directories that automatically unlink files upon parser completion.

4.4 Dual-Channel Coverage Interception

To track bandit rewards independently of libFuzzer's internal 8-bit counters, QTFuzz intercepts edge events at the linker level. We compile targets with both `-fsanitize=fuzzer` and `-fsanitize-coverage=trace-pc-guard`. Because Clang's compiler runtime provides a default implementation of `__sanitizer_cov_trace_pc_guard`, defining a duplicate symbol causes a link-time collision.

Table 1. Evaluated subject systems across target domains, test harnesses, cumulative campaign executions, and peak branch edge coverage.

Target	Domain	Version / Commit	LoC	Harnesses	Executions	Peak Cov
fastgltf	3D Asset Parser	v0.9.1 (`705564d`)	14,280	3	3,200,000	4,218
raylib	Game / Multimedia Engine	master (`e1a4079`)	118,940	37	54,320,128	47,121
drogon	Asynchronous Web Server	v1.9.8 (`df418bf`)	68,410	19	750,000,000	3,178
Total	—	—	201,630	59	807,520,128	—

We avoided this collision by using GNU linker symbol wrapping (`-Wl,--wrap=__sanitizer_cov_trace_pc_guard`). Our wrapper inspects the guard variable, increments the independent coverage table, and forwards execution to the original libFuzzer handler. We chose linker wrapping over the documented `__sanitizer_get_total_unique_coverage()` API because that API returns only a global monotonic edge count; it cannot identify which specific edge was tripped or maintain per-arm frequency ledgers. The wrapper introduces minimal runtime overhead ($< 0.8\%$) while maintaining compatibility with the sanitizer runtime.

5 Evaluation

We evaluate QTFuzz to answer four research questions:

RQ1: Throughput & Coverage: Can QTFuzz achieve high execution throughput and discover deep branch coverage across diverse domains?

RQ2: Bandit Scheduling: Does Discounted-UCB discover more branch edges than static uniform and flat byte baselines?

RQ3: Defect Discovery: What concrete vulnerabilities does QTFuzz uncover in production codebases, and are they reachable?

RQ4: Overhead & Cost: What runtime overhead do the headless shims and linker coverage interception introduce?

5.1 Subjects

We evaluate QTFuzz on three open-source C and C++ projects summarized in Table 1. First, **fastgltf** (14,280 LoC) [Spnda 2024] parses glTF 2.0 3D assets. Second, **raylib** (118,940 LoC) [Santamaria 2024] provides a game engine with graphics and audio subsystems. Third, **drogon** (68,410 LoC) [Tao 2024] is an asynchronous web framework handling HTTP and WebSockets. Together, these subjects represent 201,630 lines of code across three distinct input domains.

5.2 Baselines

We compare QTFuzz against three baselines: (i) *Flat Byte Mutator*, applying unguided random bit flips without grammar knowledge; (ii) *Static Uniform Scheduler*, executing the same four structured operators with equal, fixed probabilities (25% each) without bandit adaptation; and (iii) *Socket-Based Baseline*, driving drogon over local TCP loopback sockets (`127.0.0.1`) instead of in-memory mocks.

5.3 Setup

All experiments ran on an x86_64 workstation with an AMD processor (16 logical cores) running Ubuntu 24.04 LTS under WSL2 with 32 GB of RAM. Binaries were compiled using Clang 18.1.3 with `-fsanitize=address,undefined` and `-O2` optimization. To evaluate bandit convergence and variance on fastgltf (**RQ2**), we conducted 10 repeated trials

of 1,800 seconds (30 minutes) per configuration, following the empirical methodology of Klees et al. [Klees et al. 2018]. To discover vulnerabilities (**RQ3**), we conducted continuous 24-hour multi-worker fuzzing campaigns across the full 59-target suite.

5.4 RQ1: Throughput and Branch Coverage

Table 1 presents aggregate campaign metrics. Across all 59 target binaries, QTFuzz executed 807,520,128 inputs. On fastgltf, QTFuzz reached a peak coverage of 4,218 branch edges with a median speed of 145 executions per second in hybrid mode. On raylib, decomposing the monolithic library into 37 subsystem targets exposed deep execution paths. The fuzzer discovered 47,121 edges in the Image VM (`fuzz_image_ops`), 39,057 edges in the software rasterizer (`fuzz_render_vm`), and 22,177 edges in the glTF mesh loader (`fuzz_model_gltf`). Across all 37 harnesses, QTFuzz covered a union of 52,419 unique branch edges in the shared raylib library codebase.

Ablation: Monolithic vs. Decoupled Targets. To verify the necessity of subsystem decomposition, we executed an ablation study comparing the 37 decoupled targets against a single monolithic raylib harness multiplexing all subsystems via an initial selector byte. The monolithic harness stalled during display initialization and event loop setup, reaching only 312 executions per second and 4,110 branch edges. Decoupling subsystems into headless targets yielded a 28.5× throughput increase and a 12.7× increase in discovered unique edges.

In-Memory Mocks vs. Socket Baseline. On drogon, testing over TCP loopback sockets with fresh connections per iteration reached only 240 executions per second due to connection teardown and `TIME_WAIT` exhaustion. Under persistent keep-alive connections, loopback throughput reached 1,840 executions per second. In contrast, QTFuzz's `MockTcpConnection` achieved 53,732 executions per second on the cookie parser (a 29.2× speedup over keep-alive sockets) and 35,410 executions per second on stateful WebSocket frames, discovering 3,178 edges in `fuzz_kitchen_sink`.

Observation 1. Decoupling subsystems into headless targets enabled QTFuzz to execute over 807 million inputs, reaching up to 53,732 executions per second on network parsers without socket allocation bottlenecks.

5.5 RQ2: Adaptive Bandit Scheduling

Table 2 compares Discounted-UCB against the Static Uniform and Flat Byte baselines across 10 independent 30-minute trials on fastgltf. Discounted-UCB achieved a median of 4,218 branch edges and 428 corpus inputs. The Static Uniform scheduler, using the exact same four operators without adaptation, achieved a median of 3,845 edges (a 9.7% reduction) and 392 corpus inputs. The Flat Byte mutator achieved 3,210 edges and 375 corpus inputs. A Mann-Whitney U test confirms that Discounted-UCB discovers more edges than Static Uniform ($p = 0.0038 < 0.01$) with a large effect size (Vargha-Delaney $\hat{A}_{12} = 0.88$). The Flat Byte mutator executed 25.0% fewer inputs (197,088 vs. 262,739) because unstructured inputs frequently fail early chunk validations, exercising shallow error handlers rather than deep recursive parsing paths.

Live Arm Selection Dynamics. During live testing on drogon's `fuzz_http_request` over a 2,000-execution profiling window, the bandit evaluated four arms: `bin.libfuzzer` (650 pulls, hit rate $\bar{r} = 0.157$, discovering 482 edges across 102 coverage events); `bin.blockdup` (500 pulls, $\bar{r} = 0.095$, discovering 264 edges across 48 events); `bin.intedge` (430 pulls, $\bar{r} = 0.067$, discovering 198 edges across 29 events); and `bin.fixup` (420 pulls, $\bar{r} = 0.045$, discovering

Table 2. Evaluation of mutation schedulers across 10 repeated 30-minute trials on fastgltf (medians reported).

Scheduler Configuration	Branch Edges	Corpus Inputs	Executions	$\hat{A}_{12}$ (vs. D-UCB)
Discounted-UCB (D-UCB)	**4,218**	**428**	**262,739**	—
Static Uniform (same operators)	3,845	392	248,110	0.88 ($p < 0.01$)
Flat Byte Mutator	3,210	375	197,088	1.00 ($p < 0.001$)

Table 3. Verified upstream security defects discovered by QTFuzz across the three subject systems and their upstream resolution status.

ID	Target	Classification	CWE	Component	Trigger Condition	Upstream Status
F1	fastgltf	Out-of-Bounds Read	CWE-125	`generateMeshIndices`	Accessor index ≥ count	Closed; maintainer fixed via commit [fastgltf Maintainers 2024]
R1	raylib	Integer Underflow	CWE-191	`swPopMatrix()` in `rlsw.h`	`stackCounter == 0`	Acknowledged; PR #6362 merged [raylib Maintainers 2026a]
R2	raylib	Out-of-Bounds Read	CWE-125	`ExportAutomationEventList`	`event.type` ≥ 24	Acknowledged; maintainer fixed [raylib Maintainers 2026c]
R3	raylib	NULL Pointer Dereference	CWE-476	`LoadWaveSamples()`	Large sample allocation	Acknowledged; maintainer fixed (commit 7639166) [raylib Maintainers 2026b]
D1	drogon	Process Abort in `noexcept`	CWE-755	`HttpFileImpl::save`	Filename > 255 bytes	Closed; PR #2662 merged [Drogon Maintainers 2026b]
D2	drogon	Out-of-Bounds Iterator	CWE-125	`HttpFileImpl::save`	Traversal prefix `../../c`	Closed; maintainer fixed via commit [Drogon Maintainers 2026a]

115 edges across 19 events). Across these 198 discovery events, the bandit uncovered 1,059 new branch edges. On raylib's `fuzz_audio_wav` over 1,000 executions, the bandit allocated 580 pulls to `bin.libfuzzer` ($\bar{r} = 0.062$, 280 edges) and 420 pulls to `bin.intedge` ($\bar{r} = 0.034$, 202 edges), discovering 482 edges.

Observation 2. Discounted-UCB outperforms static uniform scheduling ($\hat{A}_{12} = 0.88, p < 0.01$), directing mutation energy to high-yield operators across distinct domain targets.

5.6 RQ3: Defect Discovery

Table 3 summarizes the six upstream-confirmed defects discovered by QTFuzz across the three codebases.

Defect F1 (fastgltf): Out-of-Bounds Heap Read (CWE-125). In `fastgltf::Parser::generateMeshIndices`, when generating indices from vertex accessors for meshes lacking index buffers, the parser read accessor indices without checking bounds against the total accessor count. A malformed glTF file triggered an out-of-bounds heap read under AddressSanitizer. The maintainer verified our report and committed a fix checking accessor bounds in Issue #160 [fastgltf Maintainers 2024].

Defect R1 (raylib): Matrix Stack Integer Underflow (CWE-191). In raylib's software rasterizer (`rlsw.h`), calling `swPopMatrix()` when `stackCounter == 0` caused an unsigned integer underflow to `UINT_MAX`. Subsequent matrix operations accessed unmapped memory via the corrupted pointer, causing a segmentation fault. This is reachable via 3D models with imbalanced transform hierarchies. The maintainer acknowledged Issue #6361 and merged Pull Request #6362 [raylib Maintainers 2026a].

Defect R2 (raylib): Out-of-Bounds Read in Event Export (CWE-125). In `src/rcore.c`, `ExportAutomationEventList()` indexes a table of 24 string pointers (`autoEventTypeName`) with unvalidated `event.type` values. Reading an untrusted `.rae` file with type ≥ 24 caused out-of-bounds memory reads. The maintainer acknowledged Issue #6378 and committed a fix [raylib Maintainers 2026c].

Defect R3 (raylib): NULL Pointer Dereference in Audio Allocation (CWE-476). In `src/raudio.c`, `LoadWaveSamples()` failed to check whether `RL_MALLOC()` returned NULL when parsing an exorbitant sample count, immediately dereferencing address zero during format conversion. The maintainer acknowledged Issue #6373 and unified sample allocation using `RL_CALLOC` in commit 7639166 [raylib Maintainers 2026b].

Defect D1 (drogon): Process Abort in `noexcept` (CWE-755). As shown in Figure 1, `HttpFileImpl::save()` was marked `noexcept`. When an application handler saves an upload using a client-controlled filename exceeding 255 bytes, `std::filesystem::exists` threw `std::filesystem_error`. The enclosing `noexcept` boundary caused `std::terminate()` to abort the multi-threaded web server process. We reproduced this end-to-end against a production server over a real TCP socket via `curl`. The maintainers merged Pull Request #2662, adopting non-throwing overloads [Drogon Maintainers 2026b].

Defect D2 (drogon): Out-of-Bounds Iterator Dereference (CWE-125). In `HttpFileImpl::save()`, path traversal checks compared upload paths against the target directory using a three-iterator loop. For filenames with traversal prefixes such as `../../c`, the loop advanced `++it1` without checking against `end()`, dereferencing out-of-bounds heap memory in release builds. We submitted Pull Request #2656, after which the maintainer committed the bounds check [Drogon Maintainers 2026a].

Additional Unfiled Medium- and High-Severity Defects. In addition to the six upstream-confirmed defects, QTFuzz uncovered ten medium- and high-severity defects verified with standalone reproducers. In raylib, we uncovered two very high-severity heap buffer overflows: an arbitrary write past allocated bone transformation matrices in `UpdateModelAnimationBones()` (CWE-122) and an audio stream buffer overflow of up to 76.8 KB in `UpdateAudioStreamInLockedState()` (CWE-122). We identified an out-of-bounds heap read in glTF accessor buffer views (CWE-125) and an unchecked mixer callback NULL dereference in `ReadAudioBufferFramesInInternalFormat()` (CWE-476). Additional findings include an integer underflow in `ImageBlurGaussian()` (CWE-191) and a backward scan buffer underflow in `GetCodepointPrevious()` (CWE-191). In drogon, QTFuzz detected premature header termination causing body desynchronization and request smuggling in `HttpRequestParser.cc` (CWE-444, verified using a differential oracle against Node.js `llhttp`). We also found out-of-range enum loads in `HttpResponseParser.cc` (CWE-758), multipart boundary semantic desynchronization (RFC 2046), and header null-byte injection in `HttpUtils.cc` (CWE-158).

Observation 3. QTFuzz uncovered 16 security defects across three codebases: six upstream-confirmed defects (two merged pull requests and four maintainer commits), alongside ten unfiled medium- and high-severity findings.

5.7 RQ4: Overhead and Cost

We measured the cost of our linker symbol wrapping and headless mocking architecture. GNU linker symbol wrapping (`-Wl,--wrap`) intercepts edge events with an atomic counter test before forwarding execution. In microbenchmarks, the throughput difference between wrapped and native libFuzzer execution was below 0.8%, within measurement noise. In terms of memory, each `MockTcpConnection` instance allocates 4.2 KB of mock state in memory, compared to operating system socket buffers that consume up to 128 KB per open descriptor. Memory footprint remained stable under 250 MB RSS across continuous 24-hour fuzzing campaigns on all 59 targets.

Observation 4. Linker-wrapped coverage interception imposes less than 1% runtime overhead, and in-memory mock connections consume 4.2 KB per instance, maintaining stable memory consumption below 250 MB RSS during continuous testing.

6 Discussion

Why did mutation operators apply across diverse domains? At first glance, 3D mesh decoding, game rendering, and HTTP web serving appear unrelated. However, their vulnerability profiles share common mechanisms: all three process hierarchical data with lengths, offsets, and binary buffers. An operator injecting integer boundaries into glTF buffer views is equally effective at triggering integer overflows in raylib audio or drogon `Content-Length` headers. Our results show that boundary value and chunk mutators find utility across domains when paired with an adaptive bandit that learns arm weights online.

Do headless in-memory shims miss real bugs? By abstracting sockets and displays, headless shims bypass kernel TCP stacks, TLS handshakes, connection reuse state machines, and hardware timing. They cannot detect kernel race conditions, TLS bugs, or GPU driver faults. However, user-space memory corruptions and parser desynchronizations reside in application code. Testing in memory enables hundreds of millions of iterations that are computationally impractical over loopback sockets.

Why are `noexcept` filesystem operations a severe threat to web server reliability? C++ developers mark file methods `noexcept` for interface contracts or exception safety. However, filesystem calls throw `std::filesystem_error` on unexpected conditions, such as filenames exceeding length limits. When an exception escapes a `noexcept` boundary, the runtime aborts via `std::terminate()`. Because web servers run worker threads in a single address space, one malformed upload crashes the entire service. Developers must use non-throwing `std::error_code` overloads for untrusted inputs.

Could static analysis have detected defect D1? Clang-Tidy provides a check (`bugprone-exception-escape`) to flag `noexcept` functions invoking throwing calls. However, in large codebases with filesystem wrappers and templates, static checkers produce high false alarms or require whole-program call graphs. In contrast, coverage-guided fuzzing produced a concrete crashing reproducer with zero false positives.

We leave the automated synthesis of headless mock shims from interface definitions for future work.

6.1 Threats to Validity

Internal Validity. Fuzz testing is non-deterministic. We mitigated this by executing 10 independent trials on fastgltf reporting Mann-Whitney U tests and Vargha-Delaney $\hat{A}_{12}$ effect sizes, running 804M inputs on raylib and drogon, and verifying all defects with standalone C++ reproducers.

External Validity. Our evaluation examines three mature C/C++ projects (fastgltf, raylib, and drogon) spanning distinct domains: binary asset parsing, multimedia rendering, and high-concurrency web servers. While diverse, our findings may not generalize to embedded firmware or safety-critical operating systems.

Construct Validity. To ensure branch edge coverage reflects testing effectiveness, we paired coverage feedback with AddressSanitizer and UndefinedBehaviorSanitizer instrumentation, confirming security impact through merged pull requests and official bug trackers.

7 Related Work

Structure-Aware and Grammar Fuzzing. Coverage-guided fuzzers such as libFuzzer [Serebryany 2016] and AFL++ [Fioraldi et al. 2020] support custom mutators for complex formats, and engines like NAUTILUS [Aschermann et al. 2019] generate grammar-valid inputs. We share their goal of maintaining structural validity to reach deep code paths. However, prior grammar fuzzers target file parsers; QTFuzz extends structure-aware mutation to multimedia rendering and network streams via headless in-memory shims.

Adaptive Mutation and Bandit Scheduling. Adaptive fuzzers dynamically allocate mutation energy: AFLFast [Böhme et al. 2016] models seed transitions via Markov chains, Woo et al. [Woo et al. 2013] formulated operator scheduling as a bandit problem, MOpt [Lyu et al. 2019] uses particle swarms, and EcoFuzz [Yue et al. 2020] applies adversarial bandits. We share their goal of optimizing mutation operator payoffs. We differ by implementing Discounted-UCB [Garivier and Moulines 2011] for non-stationary feedback and decoupling coverage accounting via GNU linker wrapping (`-Wl,--wrap`) to intercept edge rewards independently without colliding with libFuzzer.

Network and Protocol Fuzzing. Stateful network fuzzers like AFLNet [Pham et al. 2020] send packets over OS sockets, while snapshot tools like Nyx-Net [Schumilo et al. 2021] and SnapFuzz [Veen et al. 2022] accelerate fuzzing via virtualization or library shims. We share their goal of discovering protocol desynchronization and memory corruptions. However, hypervisor snapshots require virtualization, and socket fuzzing incurs overhead. We eliminate socket overhead via user-space in-memory mocks and tail slicing, reaching up to 53,732 executions per second.

8 Conclusion

Modern software stacks integrate diverse input formats, from 3D models to graphics rendering and asynchronous network requests. Testing these layered architectures with flat fuzzers leads to low throughput and missed defects from environment dependencies. We presented QTFuzz, an adaptive fuzzing architecture combining structure-aware mutation, a Discounted-UCB bandit scheduler, and headless in-memory shims. Across 59 target binaries in fastgltf, raylib, and drogon, QTFuzz executed 807,520,128 inputs, uncovering 16 security defects: six upstream-confirmed defects (two merged pull requests and four maintainer commits) alongside ten unfiled medium- and high-severity findings. Decoupling coverage accounting via linker wrapping and isolating subsystems into in-memory mocks enables high-throughput testing across diverse software domains. **Data Availability.** The replication package for QTFuzz is available at https://github.com/PASS-IITG/cs5187-term-project-shadye-6, containing harnesses, schedulers, shims, reproducers, seed corpora, and scripts.

Use of Generative AI. A generative AI assistant helped format LaTeX tables and check grammar. The authors verified all statements, equations, citations, and measurements.

References

Cornelius Aschermann, Tommaso Frassetto, Thorsten Holz, Patrick Jauernig, Ahmad-Reza Sadeghi, and Daniel Teuchert. 2019. NAUTILUS: Fishing for Deep Bugs with Grammars. In *2019 Network and Distributed System Security Symposium (NDSS)*. doi:10.14722/ndss.2019.23351

Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. 2002. Finite-time Analysis of the Multiarmed Bandit Problem. *Machine Learning* 47, 2-3 (2002), 235–256. doi:10.1023/A:1013689704352

Marcel Böhme, Van-Thuan Pham, and Abhik Roychoudhury. 2016. Coverage-based Greybox Fuzzing as Markov Chain. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS '16)*. 1032–1043. doi:10.1145/2976749.2978428

Drogon Maintainers. 2026a. Drogon Pull Request #2656: Fix out-of-bounds iterator dereference in HttpFileImpl::save(). GitHub Pull Request. Issue raised; fixed in commit by maintainer after raised.

Drogon Maintainers. 2026b. Drogon Pull Request #2662: Process termination from filesystem errors in noexcept HttpFileImpl save methods. GitHub Pull Request. Closed; PR merged.

fastgltf Maintainers. 2024. fastgltf Issue #160: Out-of-bounds heap read in Parser::generateMeshIndices. GitHub Issue Report. Closed; fixed through commit by maintainer.

Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. 2020. AFL++: Combining Incremental Steps of Fuzzing Research. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*.

Aurélien Garivier and Eric Moulines. 2011. On Upper-Confidence Bound Policies for Non-Stationary Bandit Problems. In *Algorithmic Learning Theory (ALT '11)*. 174–188. doi:10.1007/978-3-642-24412-4_16

George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. 2018. Evaluating Fuzz Testing. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS '18)*. 2123–2138. doi:10.1145/3243734.3243804

Chenyang Lyu, Shouling Ji, Chao Zhang, Yuwei Li, Han-Hsuan Lee, Yu-Huan Song, and Ting Wang. 2019. MOpt: Optimized Mutation Scheduling for Fuzzing. In *28th USENIX Security Symposium (USENIX Security 19)*. 2447–2464.

Van-Thuan Pham, Marcel Böhme, Andrew E. Santosa, Alexandru Răzvan Căciulescu, and Abhik Roychoudhury. 2020. AFLNet: A Greybox Fuzzer for Network Protocols. In *2020 IEEE International Conference on Software Testing, Verification and Validation (ICST)*. 460–467. doi:10.1109/ICST46399.2020.00054

raylib Maintainers. 2026a. raylib Issue #6361 / Pull Request #6362: Matrix stack underflow in swPopMatrix(). GitHub Pull Request. Acknowledged; PR merged.

raylib Maintainers. 2026b. raylib Issue #6373: Null-pointer dereference on allocation failure in LoadWaveSamples(). GitHub Issue Report. Acknowledged; PR closed, maintainer fixed in commit 7639166.

raylib Maintainers. 2026c. raylib Issue #6378: Out-of-bounds read in ExportAutomationEventList() with invalid event types. GitHub Issue Report. Acknowledged; maintainer fixed.

Ramon Santamaria. 2024. raylib: A simple and easy-to-use library to enjoy videogames programming. GitHub Repository. https://github.com/raysan5/raylib.

Sergej Schumilo, Cornelius Aschermann, Andrea Jemmett, Ali Abbasi, and Thorsten Holz. 2021. Nyx-Net: Network Fuzzing with Snapshot-based Virtualization. In *30th USENIX Security Symposium (USENIX Security 21)*. 2821–2838.

Konstantin Serebryany. 2016. Continuous Fuzzing with libFuzzer and AddressSanitizer. In *2016 IEEE Cybersecurity Development (SecDev)*. 157–157. doi:10.1109/SecDev.2016.043

Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov. 2012. AddressSanitizer: A Fast Address Sanity Checker. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)*. 301–308.

Alexander Spnda. 2024. fastgltf: Fast C++17 glTF 2.0 parsing and serialization library. GitHub Repository. https://github.com/spnda/fastgltf.

An Tao. 2024. Drogon: A C++17 based fast and high-performance HTTP web framework. GitHub Repository. https://github.com/drogonframework/drogon.

The Khronos Group. 2021. glTF 2.0 Specification. Specification Document. Khronos 3D Formats Working Group.

Brian Vander Veen, Enes Göktaş, Cristiano Giuffrida, and Herbert Bos. 2022. SnapFuzz: High-Throughput Fuzzing of Network Applications. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2022)*. 314–326. doi:10.1145/3533767.3534382

Maverick Woo, Sang Kil Cha, Samantha Gottlieb, and David Brumley. 2013. Scheduling Black-box Mutational Fuzzing. In *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security (CCS '13)*. 511–522. doi:10.1145/2508859.2516736

Tai Yue, Pengfei Wang, Yong Tang, Enze Wang, Bo Yu, Kai Lu, and Xu Zhou. 2020. EcoFuzz: Adaptive Energy-Saving Greybox Fuzzing as a Multi-Armed Bandit. In *29th USENIX Security Symposium (USENIX Security 20)*. 1565–1582.